

Student Name: Enow Joshua
University :
Major : Cyber Security
Internship Duration : April 10th, 2025-May 3rd, 2025
Company: Hack Security
Mentor: Mr Nishant Prajapati
Assistant Mentor: Mr Aman Pandey
Cordinator: Mr Shivam Kapoor

Objectives

My primary objective of this internship were to:

- 1) Develop a deep understanding of cybersecurity principles and practices.
- 2) Gain hands-on experience in identifying, analyzing and mitigating security threats.
- 3) Enhance my skills in using cybersecurity tools and techniques in real-world scenarios.

Task and Responsibilities

During my internship, I was involved in the following key tasks:

- **Vulnerability Assessment:** Conducted a thorough scan of a target website to identify open ports and potential vulnerabilities.
- **Penetration Testing:** Performed brute-force attacks and directory enumeration on the website to uncover hidden directories and files.
 - **Traffic Analysis:** Intercepted network traffic using Wireshark during a simulated login attempt, successfully capturing and analyzing transmitted credentials.
- **Decryption and Cryptanalysis:** Decrypted password-protected files using cryptographic tools and analyzed encoded hash values to recover plaintext passwords.
- **Reverse Engineering:** Used PE Explorer to analyze an executable file, identifying the entry point and other critical information.
- **Network Security:** Executed a de-authentication attack on a controlled network environment, capturing the handshake and subsequently cracking the Wi-Fi password using a custom wordlist.
- **Payload Creation:** Developed and deployed a Metasploit payload to establish a reverse shell connection on a virtual machine.

Internship Task

1. Find all the ports that are open on the website <http://testphp.vulnweb.com/>

On the website, I discovered port 80 (tcp) is the open port. Bellow is a clear diagram of the command used in scanning for open port and the result that was gotten from the scan. Namp -p - <http://testphp.vulnweb.com/>. This is used to carryout a full scan.

```

root@dante-Latitude-E7450: /home/dante
root@dante-Latitude-E7450: /home/dante
root@dante-Latitude-E7450: /home/dante

sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 44.228.249.3, 16) => Network is unreachable
Offending packet: TCP 192.168.1.193:50262 > 44.228.249.3:1027 S ttl=49 (id=27559 lplen=44 seq=3338413231 win=1024 <ms 1460>
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 44.228.249.3, 16) => Network is unreachable
Offending packet: TCP 192.168.1.193:50261 > 44.228.249.3:1046 S ttl=52 (id=11314 lplen=44 seq=3338478760 win=1024 <ms 1460>
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 44.228.249.3, 16) => Network is unreachable
Offending packet: TCP 192.168.1.193:50262 > 44.228.249.3:1046 S ttl=48 (id=43764 lplen=44 seq=3338413231 win=1024 <ms 1460>
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 44.228.249.3, 16) => Network is unreachable
Offending packet: TCP 192.168.1.193:50261 > 44.228.249.3:1060 S ttl=56 (id=36830 lplen=44 seq=3338413231 win=1024 <ms 1460>
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 44.228.249.3, 16) => Network is unreachable
Offending packet: TCP 192.168.1.193:50262 > 44.228.249.3:1060 S ttl=56 (id=36830 lplen=44 seq=3338413231 win=1024 <ms 1460>
sendto in send_ip_packet_sd: sendto(5, packet, 44, 0, 44.228.249.3, 16) => Network is unreachable
Offending packet: TCP 192.168.1.193:50261 > 44.228.249.3:1060 S ttl=56 (id=36830 lplen=44 seq=3338413231 win=1024 <ms 1460>
Stats: 0:01:58 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Finishing: About 40.76% done; ETC: 18:49 (6:02:52 remaining)
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.31s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered ports
PORT      STATE SERVICE
80/tcp    open  http
| http-title: Home of Acunetix Art
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: Cretion IPPanel control system (90%), Openmrt KanKaze 7.00 (Linux 2.6.22) (60%), Openmrt 0.9 - 7.00 (Linux 2.4.30 - 2.4.34) (88%), Openmrt White Russian 0.9 (Linux 2.4.30) (88%), A
SUS RI-NSOU MAP (Linux 3.4) (87%), Linux 3.1 (87%), Linux 3.10 (87%), Linux 3.2 (87%), HP P2000 G3 NAS device (87%), AXIS 210A or 211 Network Camera (Linux 2.6.17) (87%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 24 hops

TRACEROUTE (using port 80/tcp)
HOP RTT ADDRESS
1 4.47 ms _gateway (192.168.1.1)
2 ...
3 29.33 ms 10.0.23.97
4 ... 6
7 32.67 ms 10.0.23.92
8 ... 0
10 20.99 ms host-196.202.236.220.mtn.cn (196.202.236.220)
11 22.43 ms host-154.76.126.102.mtn.cn (154.76.126.102)
12 131.79 ms 41.181.247.106
13 132.22 ms 41.181.244.166
14 143.66 ms lx-bundle-21-qcore2-ldn-london.as6453.net (195.219.17.174)
15 363.65 ms lf-bundle-19-2-qcore2-nto-newyork.as6453.net (64.86.143.82)
16 310.40 ms 63.243.218.8
17 ...
18 297.21 ms lf-ae-20-2-tcore2-svl-santaclara.as6453.net (66.198.101.132)
19 291.98 ms lf-ae-0-2-tcore1-svl-santaclara.as6453.net (63.243.251.1)
20 292.22 ms lf-ae-20-3-tcore1-00s-seattle.as6453.net (63.243.225.54)
21 ... 23
24 340.39 ms ec2-44-228-249-3.us-west-2.compute.amazonaws.com (44.228.249.3)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 307.58 seconds
root@dante-Latitude-E7450: /home/dante

```

2. Brute force the website <http://testphp.vulnweb.com/> and find the directories that are present in the website.

My task was successfully carried out and came out with some findings which will be seen on a screenshot and some explanation of the screenshot.

```

This XML file does not appear to have any style information associated with it. The document tree is shown below.
<?xml version="1.0"?>
<cross-domain-policy>
  <script id="HdxVideoBootstrapper"/>
  <script id="CitrixBcrApi"/>
  <allow-access-from domain="*" to-ports="*" secure="false"/>
</cross-domain-policy>

```

Finding:
I discovered a misconfigured crossdomain.xml file at [path you found], which contains:

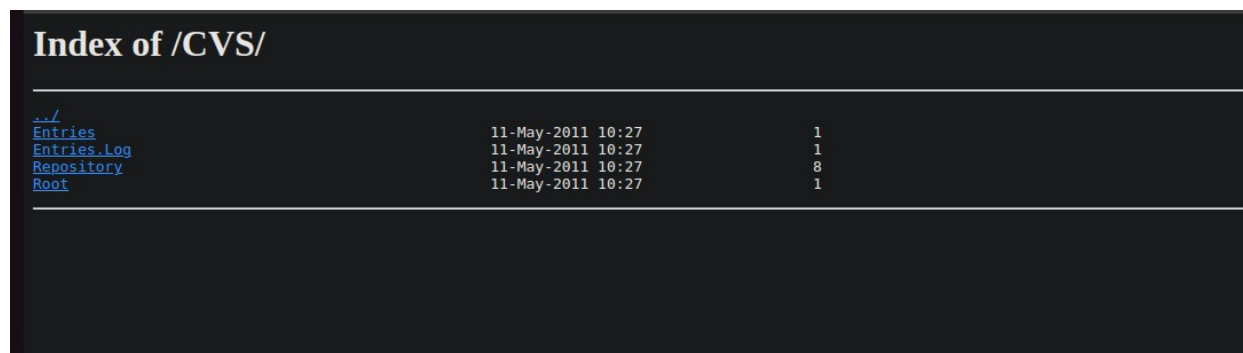
- Wildcard domain access (*)
- Access to all ports (*)
- Secure flag set to false

Implication:
This misconfiguration allows any external domain to access resources on the server via Flash or similar technologies, creating an avenue for cross-domain attacks and

potential data exfiltration.

Mitigation Recommendation:

- Limit <allow-access-from> to only trusted domains.
- Avoid wildcard domain and port usage.
- Set secure="true" to restrict to HTTPS connections.
- If Flash is not used, remove the file entirely.



Index of /CVS/		
./		
Entries	11-May-2011 10:27	1
Entries.log	11-May-2011 10:27	1
Repository	11-May-2011 10:27	8
Root	11-May-2011 10:27	1

The screenshot above shows an open CVS directory (/CVS/) with index browsing enabled. Here's what that means and why it's important:

What is CVS?

CVS stands for Concurrent Versions System. It's an old version control system used by developers to track changes in source code.

When you find a /CVS/ directory publicly accessible, it often contains internal development metadata that was accidentally uploaded to the web server.

Contents Explained:

- Entries: Lists files in the directory and their revision numbers.
- Entries.log: Contains a log of modifications made to files.
- Repository: Path to the repository – can reveal internal folder structure.
- Root: Path to the CVS repository server – may expose internal network or server info.

Security Implications:

1. Information Disclosure:
 - Reveals internal structure of the web application or repository.
 - May leak filenames, paths, or even sensitive business logic files.
2. Reconnaissance for Attackers:
 - Attackers can identify file names or versions of components to target.
 - Can lead to exploitation of old or vulnerable files not directly linked in the UI.
3. Potential Code Leakage:

- If this directory contains .php, .conf, .env, or source files in version control, you could possibly retrieve the application logic or credentials.

What to Do for Your Report:

Sample Write-up:

Finding:

Risk Level:

Impact :

Recommendations:

Apache)

```

Updated: 0 newly installed, 0 to remove and 12 not upgraded.
dante@dante-Latitude-E7450:~$ dirb http://testphp.vulnweb.com/

-----
DIRB v2.22
By The Dark Raver

-----

START_TIME: Wed Apr 16 19:12:56 2025
URL_BASE: http://testphp.vulnweb.com/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://testphp.vulnweb.com/ ----
==> DIRECTORY: http://testphp.vulnweb.com/admin/
+ http://testphp.vulnweb.com/cgi-bin (CODE:403|SIZE:276)
+ http://testphp.vulnweb.com/cgi-bin/ (CODE:403|SIZE:276)
+ http://testphp.vulnweb.com/crossdomain.xml (CODE:200|SIZE:224)
==> DIRECTORY: http://testphp.vulnweb.com/CSV/
+ http://testphp.vulnweb.com/CSV/Entries (CODE:200|SIZE:1)
+ http://testphp.vulnweb.com/CSV/Repository (CODE:200|SIZE:8)
+ http://testphp.vulnweb.com/CSV/Root (CODE:200|SIZE:1)
+ http://testphp.vulnweb.com/Favicon.ico (CODE:200|SIZE:894)

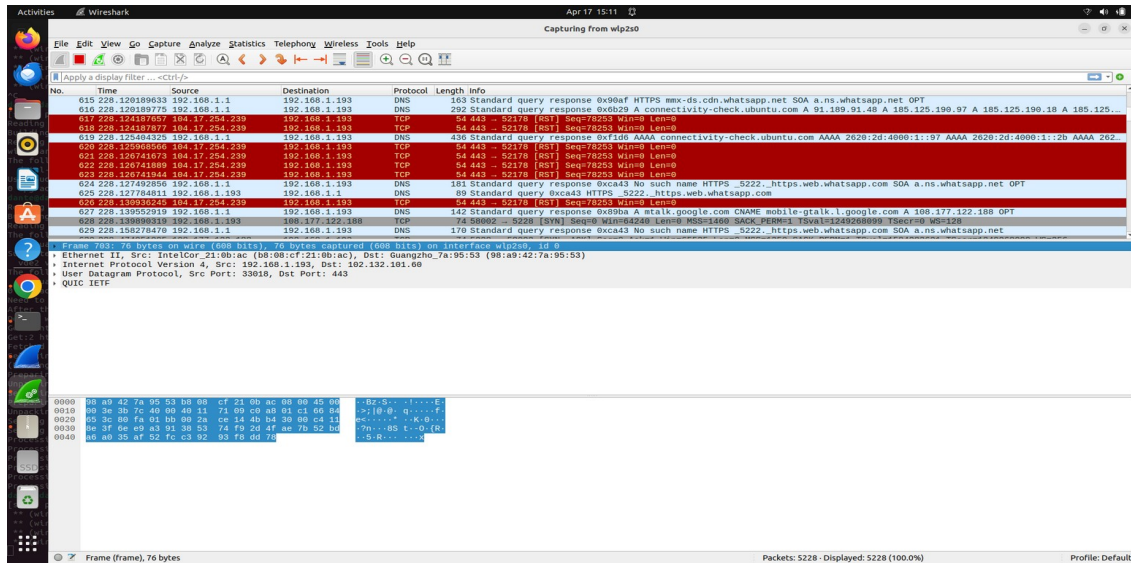
(!) FATAL: Too many errors connecting to host
(Possible cause: COULDNT CONNECT)

-----

END_TIME: Wed Apr 16 19:32:07 2025
DOWNLOADED: 1854 - FOUND: 7
dante@dante-Latitude-E7450:~$

```

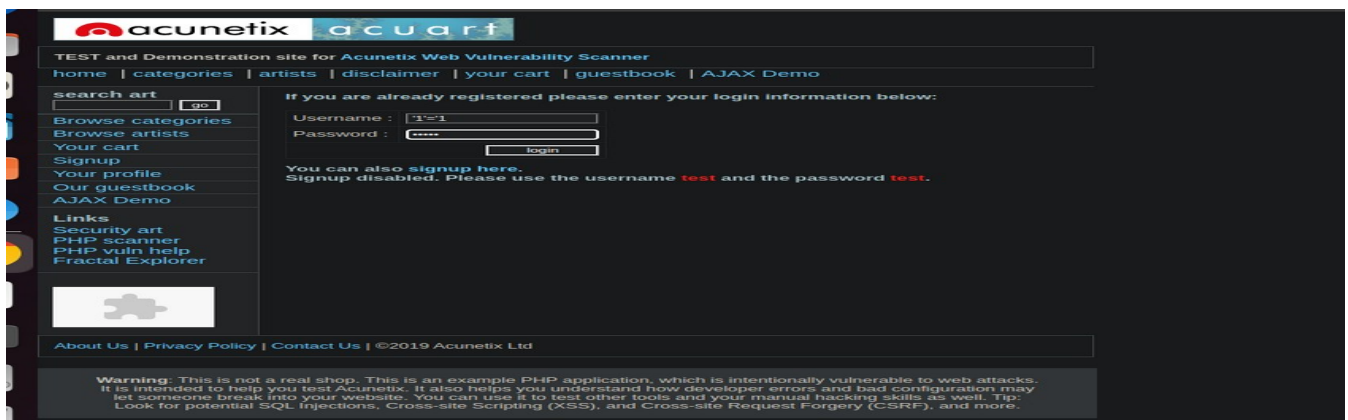
3. Make a login in the website <http://testphp.vulnweb.com/> and intercept the network traffic using wireshark and find the credentials that were transferred through the network.



Both user
name and
passwords
have been

transmitted through this protocol.

4. Perform SQL injection on the login or search page of <http://testphp.vulnweb.com/> and check if the website is vulnerable to SQLi by extracting database information. I was able to lounge a syntax where I discovered the following errors and what they mean.



You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '1'='1' AND pass='admin'" at line 1

- This shows that the SQL query you're trying to inject is malformed.
 - The snippet `'1'='1' AND pass='admin'` was likely inserted into a vulnerable SQL input field.
 - The error suggests that your injected string **broke the original query**.
-

Why I am Seeing This Error

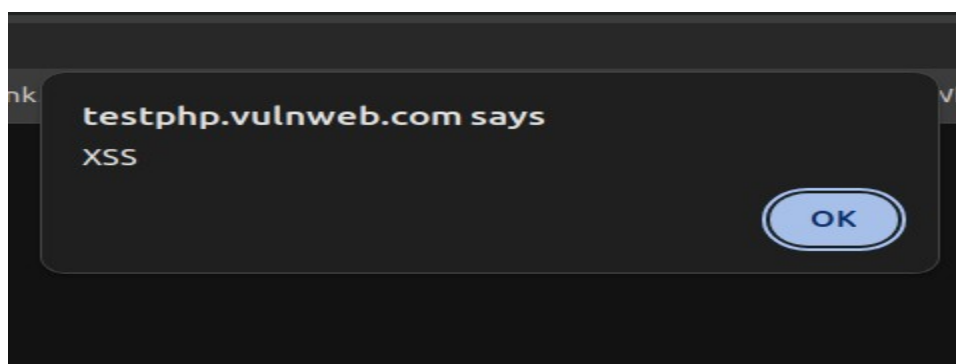
This usually happens when:

1. The query you're injecting into isn't being terminated correctly (e.g., missing `'` or `"`).
2. You're injecting into a field but not correctly balancing quotes or SQL syntax.
3. The application **isn't vulnerable** or is doing some form of input sanitization.

Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '1'='1' AND pass='admin" at line 1

5. Inject malicious JavaScript payloads in input fields (such as the comment section or search box) to see if the website is vulnerable to stored or reflected XSS attacks.

I was able to inject this malicious javascript code `<script>alert('XSS')</script>` to the website which gave me the result which will be seen on the screenshot



The screenshot is a pop-up alert box from the website testphp.vulnweb.com saying "XSS". This implies that the site has a Cross-Site Scripting (XSS) vulnerability. XSS is a vulnerability that allows an attacker to inject dangerous scripts into web pages visited by other users. In this case, the alert box is one common way of demonstrating that any JavaScript code can be executed in the browser and hence demonstrating the presence of the vulnerability.

Security Impacts

Attackers can leverage XSS vulnerabilities to steal cookies, session tokens, or other sensitive information, impersonate users, deface websites, or lead users to malicious sites.

This vulnerability compromises user data as well as the integrity of the vulnerable web application.

Recommendations

To avert and contain XSS vulnerabilities, the following are recommended:

Input Validation and Sanitization: Always validate and sanitize user input on both client-side and server-side. Strip or encode special characters that may be interpreted as code.

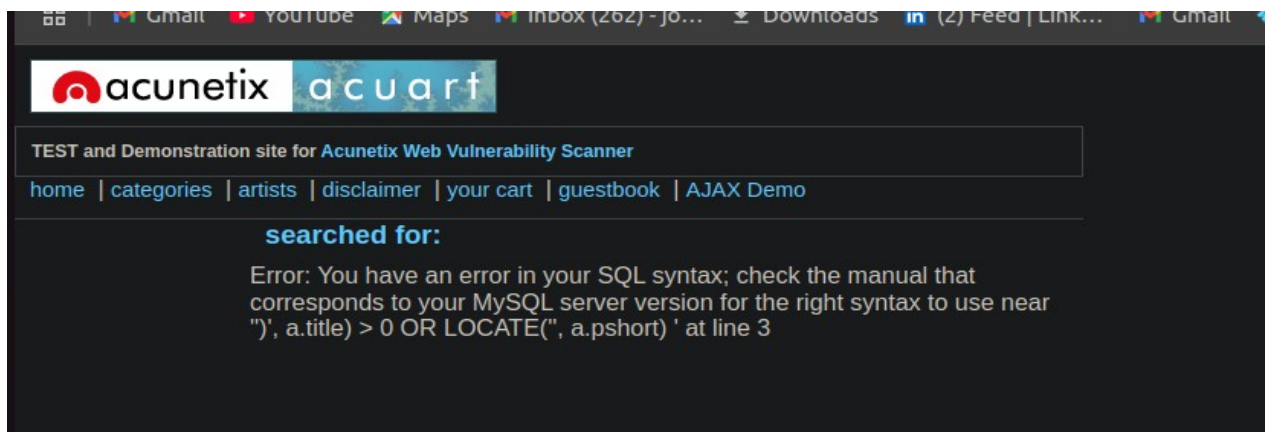
Output Encoding: Encode data before sending it to the browser, especially in HTML, JavaScript, and CSS environments.

Use Security Libraries/Frameworks: Use libraries or frameworks that have built-in defense against XSS.

Content Security Policy (CSP): Implement a strong Content Security Policy to restrict the sources upon which scripts can be executed.

Periodic Security Testing: Regularly scan web applications for XSS and other issues with automated scanners and manual testing.

By implementing these recommendations, XSS attacks can be significantly reduced, protecting both the website and its users from harm.



The following is a web application error message (Acunetix test site) of a SQL syntax error:

The error message:

"Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near."

This is the message shown when the application tries to execute a malformed SQL query. The error is reported to the user, revealing details about the database schema and query. This typically occurs when user input is inserted into a SQL query without validation or sanitization.

This is a common indicator of SQL Injection vulnerability. The attacker might have provided a specially crafted input to interfere with the SQL query such that the application displays this error. These vulnerabilities can assist attackers in tampering with database queries, hijacking sensitive data, or even destroying the entire database.

Recommendations

To correct and prevent SQL Injection vulnerabilities:

Use Prepared Statements/Parameterized Queries: Insert user input directly into SQL statements and always employ prepared statements or parameterized queries. This does not enable attackers to modify the query structure.

Input Validation and Sanitization: Sanitize and validate all user input for accepting only the expected data format and type.

Error Handling: Do not show detailed database error messages to users. Display generic error messages and on the server side, log detailed errors to use for debugging.

Least Privilege Principle: Ensure the database user account used by the application has the least privileges necessary.

Regular Security Testing: Perform regular security testing, both automated and manual, to detect and remediate SQL injection vulnerabilities.

Learning Outcomes

- **Technical Proficiency:** I gained practical experience in various cybersecurity tools and techniques, including vulnerability scanning, penetration testing, and cryptographic analysis.
- **Understanding of Cybersecurity Lifecycle:** I developed a comprehensive understanding of the steps involved in securing systems, from initial vulnerability assessment to implementing mitigations.
- **Problem-Solving Skills:** Tackling complex security challenges enhanced my analytical thinking and my ability to devise effective solutions under pressure.
- **Professional Development:** My experience improved my ability to work in a team, communicate technical information clearly, and manage time efficiently in a fast-paced environment.

Challenges and Solutions

- **Adapting to Complex Tools:** Initially, mastering advanced cybersecurity tools like Metasploit and Wireshark was challenging. I overcame this by dedicating time to study tutorials and practicing in a simulated environment, which significantly improved my proficiency.
- **Handling Advanced Security Scenarios:** The complexity of real-world security threats required a deep understanding of underlying principles. I tackled this by engaging in continuous learning and seeking guidance from my coordinator and team members.

Conclusion

My internship at Hack Secure was an enriching experience that significantly expanded my knowledge and skills in cybersecurity. The practical exposure to real-world

security challenges
and the application of advanced tools have solidified my interest in pursuing a
career in
cybersecurity. This experience has been instrumental in preparing me for the
complexities of the
cybersecurity field.

Acknowledgments

I express my sincere gratitude to Hack Secure, especially my mentor, Mr. Aman Pandey, and
assistant mentor Mr. Prabhat Raj, for their guidance and support throughout my
internship. I also
thank Amrita Vishwa Vidyapeetham for providing this internship opportunity,
which has been
crucial in my personal and professional development.

This report encapsulates the essence of my internship experience, highlighting
the integration of
academic knowledge with practical skills in a professional setting. It reflects
my journey of
learning, growth, and development in the field of cybersecurity.